

Contents

1	The One-Dimensional Tight-Binding Chain	1
1.1	Introduction	1
1.2	Tight-Binding Hamiltonian	1
1.3	Infinite Periodic Chain	2
1.4	Scattering Geometry	3
1.5	Scattering Ansatz	3
1.6	Boundary Matching	4
1.7	Linear System	4
1.8	Explicit Form of the Linear System	5
1.9	Observables	6
1.10	Physics Tests	6
1.11	From Equation to Code	7
1.12	Software Design Notes	7
1.13	Lessons Learned	8
2	Transfer Matrix Method	9
2.1	Introduction	9
2.2	Motivation	9
2.3	From the Tight-Binding Equation to a Recurrence Relation .	10
2.4	The Local Transfer Matrix	11
2.5	The Total Transfer Matrix	11
2.6	Boundary Conditions	12
2.7	Extraction of the Reflection and Transmission Amplitudes .	13
2.8	Implementation in QTransport	13
2.8.1	Implementation of the Local Transfer Matrix	14
2.8.2	Implementation of the Total Transfer Matrix	14
2.8.3	Implementation of the Boundary Vectors	15
2.8.4	Implementation of the Scattering Solver	16
2.9	Validation Against the Boundary-Matching Method	18

2.10 Numerical Results	19
2.11 Numerical Results	20

Chapter 1

The One-Dimensional Tight-Binding Chain

1.1 Introduction

The one-dimensional tight-binding chain is the simplest model that contains all the essential ingredients of a quantum scattering problem. Although simple, it already illustrates the complete workflow adopted throughout the `QTransport` project.

The purpose of this chapter is to show how a physical model is translated into a numerical solver. The path followed here is

equation $\longrightarrow$ algorithm $\longrightarrow$ code $\longrightarrow$ physics test.

The same structure will later be used for matrix-valued Hamiltonians, graphene systems, and Bogoliubov–de Gennes problems.

1.2 Tight-Binding Hamiltonian

Consider a one-dimensional lattice with nearest-neighbor hopping. The Hamiltonian is

$$H = \sum_j \epsilon_j c_j^\dagger c_j - t \sum_j \left(c_{j+1}^\dagger c_j + c_j^\dagger c_{j+1} \right), \quad (1.1)$$

where t is the hopping energy and ϵ_j is the onsite energy at site j . The stationary Schrödinger equation is

$$H|\psi\rangle = E|\psi\rangle. \quad (1.2)$$

Projecting Eq. (1.2) onto the lattice site basis gives

$$E\psi_j = \epsilon_j\psi_j - t\psi_{j-1} - t\psi_{j+1}. \quad (1.3)$$

Equivalently,

$$(E - \epsilon_j)\psi_j + t\psi_{j-1} + t\psi_{j+1} = 0. \quad (1.4)$$

Equation (1.4) is the basic equation implemented by the first version of the solver.

1.3 Infinite Periodic Chain

In the leads, the onsite energy is constant,

$$\epsilon_j = \epsilon_0. \quad (1.5)$$

For a periodic chain, Bloch solutions have the form

$$\psi_j = e^{ikj}. \quad (1.6)$$

Substituting this expression into Eq. (1.3) gives

$$Ee^{ikj} = \epsilon_0e^{ikj} - te^{ik(j-1)} - te^{ik(j+1)}. \quad (1.7)$$

Dividing by e^{ikj} ,

$$E = \epsilon_0 - te^{-ik} - te^{ik}. \quad (1.8)$$

Therefore, the dispersion relation is

$$E = \epsilon_0 - 2t \cos k. \quad (1.9)$$

Solving for k gives

$$k = \arccos\left(\frac{\epsilon_0 - E}{2t}\right). \quad (1.10)$$

The corresponding group velocity is

$$v(k) = \frac{dE}{dk} = 2t \sin k. \quad (1.11)$$

In the current implementation, this logic is represented by the class `OneDimensionalLead` in `src/qtransport/leads.py`.

1.4 Scattering Geometry

The system is divided into three regions:

1. a left semi-infinite lead;
2. a finite scattering region;
3. a right semi-infinite lead.

The scattering region contains N sites,

$$j = 0, 1, \dots, N - 1. \quad (1.12)$$

The left lead occupies the sites

$$j \leq -1, \quad (1.13)$$

and the right lead occupies the sites

$$j \geq N. \quad (1.14)$$

An incoming wave from the left lead is partially reflected and partially transmitted through the scattering region.

1.5 Scattering Ansatz

For an incoming wave from the left, the wavefunction in the left lead is

$$\psi_j = e^{ikj} + r e^{-ikj}, \quad j \leq -1, \quad (1.15)$$

where r is the reflection amplitude.

In the right lead, only an outgoing transmitted wave is allowed:

$$\psi_j = \tau e^{ikj}, \quad j \geq N, \quad (1.16)$$

where τ is the transmission amplitude.

The unknowns of the scattering problem are

$$\psi_0, \psi_1, \dots, \psi_{N-1}, r, \tau. \quad (1.17)$$

Thus the unknown vector is

$$x = (\psi_0, \psi_1, \dots, \psi_{N-1}, r, \tau)^T. \quad (1.18)$$

1.6 Boundary Matching

The Schrödinger equation is imposed at all sites in the scattering region and also at the two boundary sites adjacent to it.

For each site inside the scattering region,

$$(E - \epsilon_j)\psi_j + t\psi_{j-1} + t\psi_{j+1} = 0. \quad (1.19)$$

At the left edge of the scattering region, the site $j = 0$ is coupled to the lead site $j = -1$. From Eq. (1.15),

$$\psi_{-1} = e^{-ik} + re^{ik}. \quad (1.20)$$

At the right edge, the site $j = N - 1$ is coupled to the lead site $j = N$. From Eq. (1.16),

$$\psi_N = \tau e^{ikN}. \quad (1.21)$$

These substitutions convert the scattering problem into a finite linear system.

1.7 Linear System

The equations are assembled in the form

$$Ax = b, \quad (1.22)$$

where A is the coefficient matrix, x is the vector of unknown amplitudes, and b contains the contribution of the incoming wave.

The matrix has dimension

$$(N + 2) \times (N + 2). \quad (1.23)$$

The additional two equations come from imposing the tight-binding equation at the boundary sites $j = -1$ and $j = N$.

The numerical solution is obtained using

$$x = A^{-1}b, \quad (1.24)$$

but the code does not compute the inverse explicitly. Instead, it uses `numpy.linalg.solve`, which is more stable numerically.

1.8 Explicit Form of the Linear System

For clarity, let us write the structure of the matrix system explicitly. The unknown vector is

$$x = (\psi_0, \psi_1, \dots, \psi_{N-1}, r, \tau)^T. \quad (1.25)$$

The coefficient matrix has the form

$$A = \begin{pmatrix} t & 0 & 0 & \cdots & 0 & a_L & 0 \\ E - \epsilon_0 & t & 0 & \cdots & 0 & te^{ik} & 0 \\ t & E - \epsilon_1 & t & \cdots & 0 & 0 & 0 \\ 0 & t & E - \epsilon_2 & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & t & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & E - \epsilon_{N-1} & 0 & te^{ikN} \\ 0 & 0 & 0 & \cdots & t & 0 & a_R \end{pmatrix}, \quad (1.26)$$

where the last two columns correspond to r and τ , respectively. The boundary coefficients are

$$a_L = Ee^{ik} + te^{2ik}, \quad (1.27)$$

and

$$a_R = Ee^{ikN} + te^{ik(N+1)}. \quad (1.28)$$

The right-hand side contains only the incoming wave contribution:

$$b = \begin{pmatrix} -(Ee^{-ik} + te^{-2ik}) \\ -te^{-ik} \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}. \quad (1.29)$$

This explicit form makes the role of the boundary matching clear. The bulk rows contain the tight-binding equation inside the scattering region, while the first and last rows impose the Schrödinger equation at the boundary sites of the leads.

The important point is that the incoming wave appears in the vector b , whereas the unknown reflected and transmitted amplitudes appear as columns of the matrix A .

1.9 Observables

For identical left and right leads, the reflection and transmission probabilities are

$$R = |r|^2, \quad (1.30)$$

and

$$T = |\tau|^2. \quad (1.31)$$

Probability conservation requires

$$R + T = 1. \quad (1.32)$$

For non-identical leads, the transmission probability must include the ratio of group velocities,

$$T = \frac{v_R}{v_L} |\tau|^2. \quad (1.33)$$

This generalization will become important for graphene and Bogoliubov–de Gennes systems.

1.10 Physics Tests

The first implementation is validated using physics-based tests rather than fixed numerical values.

The tests include:

1. probability conservation, $R + T = 1$;
2. absence of reflection in a homogeneous chain;
3. partial transmission through a potential barrier;
4. rejection of energies outside the propagating band.

The homogeneous-chain test is especially important. If the scattering region is identical to the leads, then translational invariance implies

$$R = 0, \quad T = 1. \quad (1.34)$$

This test detected the first sign error in the boundary equations of the solver.

1.11 From Equation to Code

The discrete Schrödinger equation,

$$(E - \epsilon_j)\psi_j + t\psi_{j-1} + t\psi_{j+1} = 0, \quad (1.35)$$

is translated into one row of the linear system. In the implementation, the diagonal coefficient corresponds to

$$(E - \epsilon_j), \quad (1.36)$$

while the nearest-neighbor coefficients correspond to the hopping parameter t .

Thus the mathematical equation determines the structure of the matrix row assembled in the solver.

The current implementation is distributed across three files:

- `src/qtransport/leads.py`;
- `src/qtransport/solver.py`;
- `src/qtransport/observables.py`.

1.12 Software Design Notes

The first implementation introduces three basic ideas.

1. A lead is an object that knows its dispersion relation and group velocity.
2. A solver is an object that assembles and solves the boundary matching problem.
3. Observables are computed separately from the solver, so that the numerical solution and the physical interpretation remain distinct.

This separation will be preserved in later chapters. When the scalar chain is generalized to graphene or superconducting systems, the wavefunction will become a vector and the hopping constants will become matrices, but the conceptual structure will remain the same.

1.13 Lessons Learned

This first model establishes several principles that will guide the rest of the project.

- A scattering problem can be reduced to a finite linear system once the semi-infinite leads are represented analytically.
- The lead modes determine how boundary conditions are imposed.
- Physics-based tests are essential for detecting implementation errors.
- The one-dimensional chain is the minimal prototype of the general quantum transport architecture.

Chapter 2

Transfer Matrix Method

2.1 Introduction

In the previous chapter, the one-dimensional scattering problem was solved by constructing a linear system whose unknowns were the wavefunction amplitudes inside the scattering region together with the reflection and transmission amplitudes.

Although this approach is straightforward and robust, it is not the only possible formulation of the problem. An alternative consists of rewriting the discrete Schrödinger equation as a recurrence relation connecting the wavefunction at neighboring lattice sites. This reformulation naturally leads to the transfer matrix method.

The transfer matrix method is one of the most widely used techniques in quantum transport. Besides its conceptual simplicity, it provides a natural description of wave propagation through layered systems.

The main objective of this chapter is to derive the transfer matrix directly from the tight-binding equation and to demonstrate that it provides an alternative solution of exactly the same physical problem studied in Chapter 1.

Although the derivation is presented for a one-dimensional chain, the same ideas will later be generalized to matrix-valued Hamiltonians, graphene, bilayer graphene and Bogoliubov–de Gennes systems.

2.2 Motivation

The boundary-matching method solves the scattering problem by assembling a global linear system,

$$Ax = b, \quad (2.1)$$

whose dimension increases with the size of the scattering region.

The transfer matrix method follows a different philosophy. Instead of solving for all wavefunction amplitudes simultaneously, the wavefunction is propagated from one lattice site to the next by means of a local linear transformation.

The entire scattering region is therefore represented by the product of many small matrices, each describing the propagation across a single lattice site.

This approach has several advantages.

- The propagation through each site is described by the same mathematical object.
- The total transfer matrix is obtained by successive matrix multiplications.
- The formalism extends naturally to systems whose wavefunction has internal degrees of freedom.

The last point is particularly important for the `QTransport` project. In graphene, each longitudinal slice contains several internal components. Nevertheless, the transfer-matrix idea remains the same.

2.3 From the Tight-Binding Equation to a Recurrence Relation

The starting point is the one-dimensional tight-binding equation,

$$(E - \epsilon_j)\psi_j + t\psi_{j-1} + t\psi_{j+1} = 0. \quad (2.2)$$

The objective is to isolate the wavefunction at the next lattice site. Rearranging Eq. (2.2),

$$t\psi_{j+1} = -(E - \epsilon_j)\psi_j - t\psi_{j-1}. \quad (2.3)$$

Therefore,

$$\psi_{j+1} = -\frac{E - \epsilon_j}{t}\psi_j - \psi_{j-1}. \quad (2.4)$$

Equation (2.4) expresses the wavefunction at site $j + 1$ as a linear combination of the wavefunctions at the two preceding sites.

2.4 The Local Transfer Matrix

Introduce the two-component transfer vector

$$\Phi_j = \begin{pmatrix} \psi_j \\ \psi_{j-1} \end{pmatrix}. \quad (2.5)$$

Using Eq. (2.4), one obtains

$$\Phi_{j+1} = M_j \Phi_j, \quad (2.6)$$

where

$$M_j = \begin{pmatrix} -\frac{E - \epsilon_j}{t} & -1 \\ 1 & 0 \end{pmatrix}. \quad (2.7)$$

The original second-order difference equation has therefore been transformed into a first-order matrix recurrence relation.

Each lattice site is represented by a single 2×2 matrix.

Design Decision

The transfer matrix is introduced as a numerical representation of wave propagation rather than as a physical object.

Consequently, the transfer matrix is implemented in `transfer.py`, while the Hamiltonian and the scattering region remain independent objects.

This separation allows the same physical problem to be solved using different numerical techniques.

2.5 The Total Transfer Matrix

The local transfer matrix describes the propagation across a single lattice site. To determine the propagation through the entire scattering region, the local matrices are multiplied successively.

For a scattering region containing N sites,

$$\Phi_1 = M_0 \Phi_0, \quad (2.8)$$

$$\Phi_2 = M_1 \Phi_1 = M_1 M_0 \Phi_0, \quad (2.9)$$

$$\Phi_3 = M_2 \Phi_2 = M_2 M_1 M_0 \Phi_0. \quad (2.10)$$

After crossing the complete scattering region,

$$\Phi_N = M\Phi_0, \quad (2.11)$$

where

$$M = M_{N-1}M_{N-2}\cdots M_1M_0. \quad (2.12)$$

The matrix M is the total transfer matrix of the scattering region.

Physical Insight

The local transfer matrix describes propagation through a single lattice site.

The total transfer matrix describes propagation through the entire device.

This is analogous to building a complete evolution operator from many local evolution steps.

2.6 Boundary Conditions

The transfer matrix propagates the wavefunction through the scattering region. To determine the reflection and transmission amplitudes, the lead boundary conditions must still be imposed.

For an incoming wave from the left,

$$\psi_j = e^{ikj} + re^{-ikj}, \quad j \leq -1. \quad (2.13)$$

In the right lead,

$$\psi_j = \tau e^{ikj}, \quad j \geq N. \quad (2.14)$$

The transfer vectors at the two sides of the scattering region are

$$\Phi_0 = \begin{pmatrix} \psi_0 \\ \psi_{-1} \end{pmatrix}, \quad \Phi_N = \begin{pmatrix} \psi_N \\ \psi_{N-1} \end{pmatrix}. \quad (2.15)$$

Using the left-lead ansatz,

$$\Phi_0 = \begin{pmatrix} 1 + r \\ e^{-ik} + re^{ik} \end{pmatrix}. \quad (2.16)$$

Using the right-lead ansatz,

$$\Phi_N = \tau \begin{pmatrix} e^{ikN} \\ e^{ik(N-1)} \end{pmatrix}. \quad (2.17)$$

Substitution into Eq. (2.11) gives

$$\tau \begin{pmatrix} e^{ikN} \\ e^{ik(N-1)} \end{pmatrix} = M \begin{pmatrix} 1 + r \\ e^{-ik} + re^{ik} \end{pmatrix}. \quad (2.18)$$

This equation contains the complete scattering problem.

2.7 Extraction of the Reflection and Transmission Amplitudes

Define the three boundary vectors

$$\Phi_{\text{in}} = \begin{pmatrix} 1 \\ e^{-ik} \end{pmatrix}, \quad \Phi_{\text{ref}} = \begin{pmatrix} 1 \\ e^{ik} \end{pmatrix}, \quad \Phi_{\text{out}} = \begin{pmatrix} e^{ikN} \\ e^{ik(N-1)} \end{pmatrix}. \quad (2.19)$$

Then the boundary equation can be written as

$$M(\Phi_{\text{in}} + r\Phi_{\text{ref}}) = \tau\Phi_{\text{out}}. \quad (2.20)$$

Collecting the unknown amplitudes gives

$$rM\Phi_{\text{ref}} - \tau\Phi_{\text{out}} = -M\Phi_{\text{in}}. \quad (2.21)$$

Equation (2.21) is a 2×2 linear system for the reflection amplitude r and the transmission amplitude τ .

2.8 Implementation in QTransport

The transfer-matrix formalism is implemented in

`src/qtransport/transfer.py`.

The source code follows the mathematical structure derived above:

local matrix $\longrightarrow$ total matrix $\longrightarrow$ boundary vectors $\longrightarrow$ scattering amplitudes.

2.8.1 Implementation of the Local Transfer Matrix

The method `local_matrix()` implements Eq. (2.7).

```

54      Construct the local transfer matrix M_j.
55
56      The tight-binding equation
57
58       $(E - \epsilon_j) \psi_j + t \psi_{j-1} + t \psi_{j+1} = 0$ 
59
60      is rewritten as
61
62       $\psi_{j+1} = -((E - \epsilon_j)/t) \psi_j - \psi_{j-1}.$ 
63
64      Therefore,
65
66       $M_j = \begin{bmatrix} -(E - \epsilon_j)/t, & -1 \\ 1, & 0 \end{bmatrix}.$ 
67
68      """
69      hopping = self.region.hopping
70
71      return np.array(
72          [
73              [-(energy - onsite) / hopping, -1.0],
74              [1.0, 0.0],
75          ],
76          dtype=complex,
77      )
78
79      def total_matrix(self, energy: float) -> np.ndarray:

```

The matrix element

$$-\frac{E - \epsilon_j}{t}$$

appears directly in the first row of the returned NumPy array. The second row implements the shift

$$(\psi_j, \psi_{j-1}) \mapsto (\psi_{j+1}, \psi_j).$$

2.8.2 Implementation of the Total Transfer Matrix

The method `total_matrix()` implements the ordered product

$$M = M_{N-1} M_{N-2} \cdots M_0.$$

```

81     Construct the total transfer matrix of the scattering region.
82
83     If the scattering region contains N sites, then
84
85          $M = M_{\{N-1\}} M_{\{N-2\}} \dots M_1 M_0.$ 
86
87     The multiplication order matters because transfer matrices
88     generally do not commute.
89     """
90     total = np.eye(2, dtype=complex)
91
92     for onsite in self.region.onsite:
93         total = self.local_matrix(energy, onsite) @ total
94
95     return total
96
97     def incoming_vector(self, wave_number: float) -> np.ndarray:

```

The method starts from the identity matrix, corresponding to propagation through an empty region. Each local matrix is then multiplied from the left, preserving the ordering required by Eq. (2.12).

Computational Remark

For a one-dimensional chain containing N sites, the construction of the total transfer matrix requires N local matrix multiplications. The cost therefore scales linearly with the number of sites.

2.8.3 Implementation of the Boundary Vectors

The vectors Φ_{in} , Φ_{ref} and Φ_{out} are implemented by three small methods.

```

99     Return the incoming basis vector at the left boundary.
100
101      $\Phi_{\text{in}} = (1, \exp(-ik))^T.$ 
102     """
103     return np.array(
104         [1.0, np.exp(-1j * wave_number)],
105         dtype=complex,
106     )
107
108     def reflected_vector(self, wave_number: float) -> np.ndarray:
109         """
110         Return the reflected basis vector at the left boundary.
111

```

```

112     Phi_ref = (1, exp(ik))^T.
113     """
114     return np.array(
115         [1.0, np.exp(1j * wave_number)],
116         dtype=complex,
117     )
118
119     def transmitted_vector(self, wave_number: float) -> np.ndarray:
120         """
121         Return the outgoing transmitted basis vector at the right boundary.
122
123         Phi_out = (exp(ikN), exp(ik(N-1)))^T.
124         """
125         number_of_sites = self.region.size
126
127         return np.array(
128             [
129                 np.exp(1j * wave_number * number_of_sites),
130                 np.exp(1j * wave_number * (number_of_sites - 1)),
131             ],
132             dtype=complex,
133         )
134
135     def solve(self, energy: float) -> ScatteringResult:

```

Although these methods are simple, separating them from `solve()` makes the boundary conditions explicit and improves readability.

2.8.4 Implementation of the Scattering Solver

The method `solve()` constructs the final 2×2 system,

$$rM\Phi_{\text{ref}} - \tau\Phi_{\text{out}} = -M\Phi_{\text{in}},$$

and solves it using `numpy.linalg.solve()`.

```

137     Solve the scattering problem at a fixed energy.
138
139     The total transfer matrix satisfies
140
141     Phi_N = M Phi_0.
142
143     Using
144
145     Phi_0 = Phi_in + r Phi_ref
146

```

```

147         and
148
149         Phi_N = tau Phi_out,
150
151         we obtain
152
153         r M Phi_ref - tau Phi_out = - M Phi_in.
154
155         This is a 2x2 linear system for r and tau.
156         """
157         wave_number = self.left_lead.wave_number(energy)
158         total_matrix = self.total_matrix(energy)
159
160         incoming = self.incoming_vector(wave_number)
161         reflected = self.reflected_vector(wave_number)
162         transmitted = self.transmitted_vector(wave_number)
163
164         coefficient_matrix = np.column_stack(
165             (
166                 total_matrix @ reflected,
167                 -transmitted,
168             )
169         )
170
171         right_hand_side = -(total_matrix @ incoming)
172
173         reflection_amplitude, transmission_amplitude = np.linalg.solve(
174             coefficient_matrix,
175             right_hand_side,
176         )
177
178         return ScatteringResult(
179             energy=energy,
180             reflection_amplitude=reflection_amplitude,
181             transmission_amplitude=transmission_amplitude,
182         )

```

The two unknowns returned by this system are the reflection and transmission amplitudes. They are stored in a `ScatteringResult` object, which also provides the corresponding reflection and transmission probabilities.

Design Decision

The implementation separates propagation from extraction of scattering amplitudes.

The total transfer matrix describes propagation through the scattering region, while the final 2×2 linear system imposes the lead boundary conditions.

This separation will be useful when the scalar wavefunction is replaced by a vector-valued wavefunction in graphene and superconducting systems.

2.9 Validation Against the Boundary-Matching Method

Whenever a new numerical method is introduced, it is essential to verify that its implementation correctly reproduces the expected physical results. In the present work, the transfer-matrix method is validated by comparing its predictions with those obtained using the boundary-matching method developed in Chapter 1.

Although both methods originate from different mathematical formulations, they solve exactly the same scattering problem. Consequently, they must produce identical reflection and transmission probabilities for every incident energy.

Denoting the reflection and transmission probabilities obtained from the boundary-matching method by

$$R_{\text{BM}}(E), \quad T_{\text{BM}}(E),$$

and those obtained from the transfer-matrix method by

$$R_{\text{TM}}(E), \quad T_{\text{TM}}(E),$$

the following identities must hold,

$$R_{\text{BM}}(E) = R_{\text{TM}}(E), \tag{2.22}$$

$$T_{\text{BM}}(E) = T_{\text{TM}}(E). \tag{2.23}$$

These equalities provide a much stronger validation than merely checking probability conservation. Indeed, both implementations individually satisfy

$$R(E) + T(E) = 1,$$

which confirms that each numerical method conserves probability. However, probability conservation alone does not guarantee that the implemented algorithm is correct.

The comparison between two completely independent numerical methods constitutes a far more demanding validation because both algorithms would have to contain precisely the same error in order to produce the same incorrect result.

For this reason, the agreement between the two methods is adopted as the primary validation criterion throughout the development of the **QTransport** framework.

Validation Principle

Whenever possible, every new numerical method implemented in **QTransport** should be compared against an independent algorithm that solves the same physical problem.

Agreement between two independent implementations provides considerably stronger evidence of correctness than agreement with a single numerical calculation.

In the present chapter, this validation is performed by the automated test suite. The file

```
tests/physics/test_transfer_matrix.py
```

contains a test that compares the transmission probability computed with the transfer-matrix solver against the result obtained with the boundary-matching solver. The two methods agree within numerical precision for the tested scattering region.

2.10 Numerical Results

To illustrate the transfer-matrix method, we consider the transmission of an electron through a finite one-dimensional potential barrier.

The barrier consists of twenty lattice sites whose onsite energy is chosen as

$$\epsilon_j = 0.5t,$$

while the hopping parameter is kept constant throughout the system. The transmission probability is calculated by solving an independent scattering problem for each incident energy in the interval

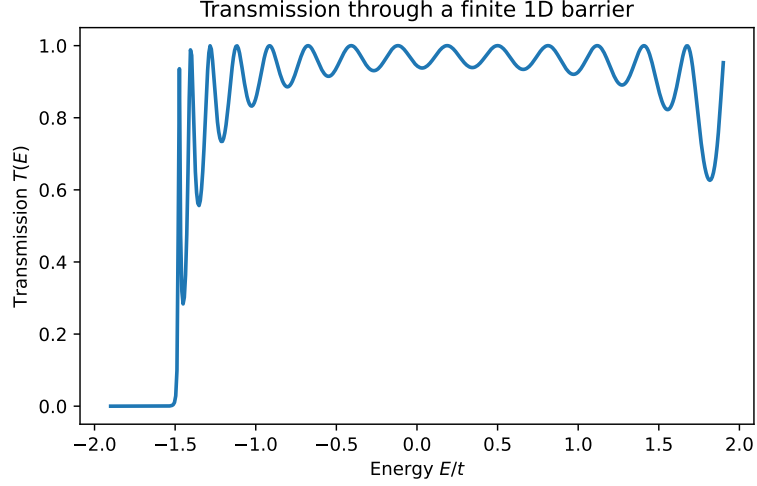


Figure 2.1: Validation of the transfer-matrix solver through the transmission probability of a finite one-dimensional barrier. The spectrum was computed using the transfer-matrix method and agrees, within numerical precision, with the boundary-matching solver introduced in Chapter 1.

$$-1.9t \leq E \leq 1.9t.$$

Figure 2.2 shows the transmission spectrum of the finite barrier. Several important features can be identified.

First, the transmission coefficient satisfies

$$0 \leq T(E) \leq 1,$$

as required by probability conservation.

Second, the transmission varies continuously with the incident energy. This behavior reflects the quantum interference produced by multiple reflections inside the finite scattering region.

Finally, the same transmission spectrum is obtained from the boundary-matching solver. The figure therefore plays two roles: it is the first numerical transmission result generated by the project, and it also validates the transfer-matrix implementation against an independent method.

Computational Experiment

System	One-dimensional tight-binding chain
Barrier length	20 lattice sites
Barrier height	$\epsilon_j = 0.5t$
Hopping	$t = 1$
Energy range	$-1.9t \leq E \leq 1.9t$
Numerical method	Transfer Matrix
Example program	<code>examples/02_transfer_matrix_barrier.py</code>
Generated figure	<code>figures/generated/02_transfer_matrix_barrier.pdf</code>

2.11 Lessons Learned

This chapter introduced the transfer-matrix method as a second, independent formulation of the one-dimensional tight-binding scattering problem.

The main mathematical lesson is that the second-order tight-binding difference equation can be rewritten as a first-order matrix recurrence relation,

$$\Phi_{j+1} = M_j \Phi_j.$$

The main numerical lesson is that the full scattering region can be represented by the ordered product of local transfer matrices,

$$M = M_{N-1} M_{N-2} \cdots M_0.$$

The main software lesson is that propagation and boundary matching should remain separate operations. In the implementation, the total transfer matrix describes propagation through the scattering region, whereas a final small linear system extracts the physical scattering amplitudes.

The main validation lesson is that probability conservation is necessary but not sufficient. A stronger validation is obtained by comparing two independent algorithms that solve the same physical problem.

Finally, this chapter establishes a pattern that will be followed throughout the rest of the project:

$$\text{derive} \longrightarrow \text{implement} \longrightarrow \text{test} \longrightarrow \text{compare} \longrightarrow \text{plot}.$$

Looking Ahead

The next step is to replace the scalar wavefunction ψ_j by a vector-valued wavefunction Ψ_j . This generalization will allow the same transfer-matrix architecture to describe systems with internal degrees of freedom, such as graphene, bilayer graphene and Bogoliubov–de Gennes Hamiltonians.

2.12 Reproducing the Results

The numerical results presented in this chapter can be reproduced from the root directory of the repository by running

The first command runs the physics tests, including the comparison between the transfer-matrix and boundary-matching solvers. The second command regenerates the transmission figure shown in Fig. 2.2.

The generated figure is written to

This reproducibility procedure closes the loop between theory, code, tests and numerical results.